CHAPTER 08

ElastiCache Redis

Caching · Session Management · Rate Limiting · Sorted Sets · Cluster Mode · Spring Boot

08-00 Chapter Overview — ElastiCache Redis

Amazon ElastiCache for Redis is a managed in-memory cache that serves data in under 1 millisecond. Unlike DynamoDB (1-3ms) or Aurora (5-50ms), Redis serves data from RAM — the fastest possible storage. ShopFlow uses Redis for: caching frequently read product details (avoid hitting Aurora for every product page load), storing user session tokens (fast validation on every API request), rate limiting (track API calls per user per minute), and the ShopFlow leaderboard of top-selling products.

■ Why Redis Matters for ShopFlow

Without Redis: every product page load triggers a SELECT from Aurora. At 1,000 requests/second, Aurora handles 1,000 queries/second — manageable, but expensive and adds 10-50ms latency. With Redis: the first request fetches from Aurora and caches in Redis for 5 minutes. The next 999 requests (in that 5-minute window) are served from Redis at < 1ms. Aurora receives 1 query per 5 minutes (12/hour) instead of 1,000/second.

Example: Product detail for iPhone 15: cached in Redis as "product:prod-iphone-15" with a 5-minute TTL. Cache hit rate: 97% (only 3% of requests miss the cache and hit Aurora). Result: Aurora handles 30 queries/second (3% of 1,000) instead of 1,000. Aurora can be a smaller instance, saving $200/month.

Use Case	Without Redis	With Redis	Latency Improvement
Product page load	50ms (Aurora query + JOIN)	< 1ms (Redis GET)	50x faster
Session token validation	5ms (Aurora SELECT by session_id)	< 1ms (Redis GET by token)	5x faster; critical for every AP
Rate limiting (API throttle)	Complex: requires Aurora transaction per request	< 1ms (Redis INCR + EXPIRE atomic operations)	Simple and fast; no Aurora in
Homepage top products	200ms (Aurora aggregation + sort)	< 1ms (Redis ZRANGE on sorted set)	200x faster; pre-computed lea

08-01 Redis Data Structures — More Than Just Cache

Redis is not just a key-value cache — it has 5 core data structures. Choosing the right structure for each use case avoids complex application logic. ShopFlow uses all five data structures for different features.

Data Structure	Redis Type	Operations	ShopFlow Use Case
String	SET key value EX seconds	GET, SET, INCR, DECR, APPEND	Product cache (JSON), session tokens,
Hash	HSET key field value	HGET, HSET, HGETALL, HMSET	User profile cache (field per attribute: na
List	LPUSH / RPUSH key value	LPOP, RPOP, LRANGE, LLEN	Recent orders queue (push new orders;
Set	SADD key member	SMEMBERS, SISMEMBER, SINTERSTORE, SUNIONSTORE	Products viewed by a user (de-duplicate
Sorted Set	ZADD key score member	ZRANGE, ZRANGEBYSCORE, ZINCRBY, ZRANK	Top 10 products by sales this week; le

08-02 Creating ElastiCache Redis in the Console

ElastiCache Redis for ShopFlow is a cluster-mode enabled setup with 3 shards across 3 Availability Zones. Cluster mode distributes data (and load) across shards using consistent hashing — each shard owns a portion of the keyspace. This allows horizontal scaling of both memory and throughput beyond what a single Redis node supports.

■ Create ShopFlow ElastiCache Redis Cluster

1. Search "ElastiCache" in the console → Redis clusters → Create Redis cluster
2. Cluster mode: Enabled (allows horizontal sharding; recommended for production)
3. Cluster name: shopflow-redis-cluster
4. Location: AWS Cloud | Multi-AZ: enabled
5. Engine version: 7.1 (latest stable)
6. Node type: cache.r6g.large (2 vCPU, 13.07 GB memory, $0.156/hr)
7. Number of shards: 3 (distributes data across 3 shards)
8. Replicas per shard: 1 (each shard has 1 primary + 1 replica = 6 nodes total)
9. Subnet group: Create new → shopflow-redis-subnet-group → select data subnets
10. Security group: shopflow-redis-sg (allows port 6379 from shopflow-ecs-sg only)
11. Encryption in transit: Yes (TLS) | Encryption at rest: Yes
12. Backup: Enable automatic backups → 3 days retention | Click Create

■ aws › ElastiCache › Redis clusters › Create Redis cluster

Cluster settings

Cluster name

shopflow-redis-cluster

Cluster mode

■ Enabled (cluster mode: data sharded across multiple nodes; can scale horizontally) ■ Disabled (single primary + replicas; scale vertically only)

■ Node type

Node type

cache.r6g.large (2 vCPU | 13.07 GB | $0.156/hr) — Graviton2; best price/perf for Redis ■

Number of shards

3

■ Each shard handles 1/3 of the keyspace (keys hash to shards by consistent hashing)

Replicas per shard

1

■ 1 replica per shard: 3 primaries + 3 replicas = 6 nodes total for HA

■ Security

■ ■ Encryption in transit (TLS) — required; Redis on plain TCP is unacceptable for production

■ ■ Encryption at rest — protects data if storage media is compromised

Create

08-03 Connecting to Redis from Java — Spring Boot

Spring Boot integrates with Redis via Spring Data Redis (Lettuce client). ShopFlow uses Lettuce (the default Spring Boot Redis client) in cluster mode, which automatically discovers all cluster nodes, distributes requests, and handles node failures transparently. The application code uses the same API whether Redis is standalone or cluster mode.

```
# application.yml — ElastiCache cluster-mode connection
spring:
data:
redis:
# Cluster mode: provide one node; Lettuce discovers the rest
cluster:
nodes:
- shopflow-redis-cluster.abc123.clustercfg.use1.cache.amazonaws.com:6379
max-redirects: 3 # max MOVED redirections for cluster key routing
ssl: true # required for ElastiCache TLS
timeout: 500ms
lettuce:
pool:
max-active: 16 # max connections per node (3 nodes × 16 = 48 total)
max-idle: 8
min-idle: 2
# Java: Use Spring Cache abstraction
@Cacheable(value = "products", key = "#productId", unless = "#result == null")
public ProductDto getProduct(String productId) {
// Called only on cache MISS (cache hit returns from Redis directly)
return productRepository.findById(productId)
.map(this::toDto)
.orElse(null);
}
@CacheEvict(value = "products", key = "#product.productId")
public ProductDto updateProduct(ProductDto product) {
// Evicts from Redis when product is updated
return productRepository.save(product);
}
```

08-04 Redis Caching Patterns

There are several caching patterns, each with different consistency and complexity trade-offs. ShopFlow uses Cache-Aside (lazy loading) for product details and Write-Through for inventory counts.

Pattern	How It Works	Consistency	ShopFlow Use
Cache-Aside (Lazy)	App checks cache → miss → fetch from DB → write to cache with TTL	Eventual (stale until TTL expires)	Product details. Cache filled on first read. Invalidated on @CacheEvict or TTL.
Write-Through	On every DB write, also write to cache simultaneously	Strong (cache always current)	Inventory counts. Updated on every stock change.
Write-Behind (Write-Back)	Write to cache first; flush to DB asynchronously in batches	Risk of data loss if cache fails before flush	Not used by ShopFlow; too risky for order and payment data.
Read-Through	Cache sits in front of DB; on miss, cache fetches from DB (not app)	Eventual (TTL-based)	DAX for DynamoDB is a read-through cache. Transparent to application.

08-05 Session Management with Redis

ShopFlow user sessions are stored in Redis. When a user logs in (via Cognito), a session token is created and stored in Redis with a 24-hour TTL. Every API request validates the token with a Redis GET (< 1ms). When the user logs out, DELETE removes the token from Redis. Redis handles millions of concurrent sessions — each session is one Redis key with ~200 bytes of data.

```
// Spring Boot: session stored in Redis via Spring Session
@EnableRedisHttpSession(maxInactiveIntervalInSeconds = 86400) // 24 hours
@Configuration
public class SessionConfig {
// Spring Session automatically:
// 1. Creates session in Redis on login
```

```
// 2. Validates session on every request (GET redis:session:SESSION_ID)
// 3. Sets TTL = 24 hours (extended on each request)
// 4. Deletes session on logout (DEL redis:session:SESSION_ID)
}
// Redis key pattern for sessions:
// spring:session:sessions:SESSION_ID → Hash: {userId, email, roles, created_at}
// spring:session:expirations:1705276800 → Set of session IDs expiring at that time
// Manual Redis session example (if not using Spring Session):
String sessionId = UUID.randomUUID().toString();
String sessionKey = "session:" + sessionId;
redisTemplate.opsForValue().set(sessionKey, userId, Duration.ofHours(24));
// Validate session (every API request):
String userId = (String) redisTemplate.opsForValue().get("session:" + sessionId);
if (userId == null) throw new UnauthorizedException("Session expired");
```

08-06 Rate Limiting with Redis

ShopFlow limits API calls to prevent abuse: 100 requests per minute per user for the order API, 1,000 per minute for product search. Redis atomic operations make rate limiting simple and fast: INCR (increment counter) and EXPIRE (set TTL) are combined in a Lua script to create a sliding window rate limiter that is atomic and sub-millisecond.

```
// Redis rate limiting — Spring Boot + Lettuce
@Service
public class RateLimiter {
@Autowired RedisTemplate redisTemplate;
// Returns true if request is allowed; false if rate limit exceeded
public boolean isAllowed(String userId, String endpoint, int maxRequests) {
String key = "rate:" + userId + ":" + endpoint + ":" + currentMinute();
// Atomic increment + set expiry (Lua script runs atomically in Redis)
Long count = redisTemplate.execute(new DefaultRedisScript<>(
"local c = redis.call("incr", KEYS[1])\n" +
"if c == 1 then redis.call("expire", KEYS[1], 60) end\n" +
"return c",
Long.class
), List.of(key));
return count != null && count <= maxRequests;
}
private String currentMinute() {
return String.valueOf(System.currentTimeMillis() / 60000);
}
}
```

■ For more sophisticated rate limiting (sliding window, token bucket), use the Redisson library which provides production-grade distributed rate limiter implementations built on Redis. Redisson's RRateLimiter handles distributed rate limiting across multiple JVM instances automatically.

08-07 Redis Top-Selling Products Leaderboard

Redis Sorted Sets maintain an ordered list of members by score. ShopFlow uses a Sorted Set for the "Top Selling Products This Week" leaderboard — updated in real-time as orders are placed and served to every homepage visitor in microseconds, without any database query.

```
// Redis Sorted Set: top products leaderboard
// Key: "leaderboard:weekly:2024-W46"
// Score = total units sold this week
// Member = product_id
// When an order is placed: increment the product score
@EventListener(OrderPlacedEvent.class)
public void onOrderPlaced(OrderPlacedEvent event) {
for (OrderItem item : event.getItems()) {
String key = "leaderboard:weekly:" + currentWeek();
```

```
// ZINCRBY key increment member — atomic; thread-safe
redisTemplate.opsForZSet().incrementScore(key, item.getProductId(), item.getQuantity());
// Set TTL to 2 weeks (auto-expire old leaderboards)
redisTemplate.expire(key, Duration.ofDays(14));
}
}
// Get top 10 products (homepage): ZRANGE key 0 9 REV WITHSCORES
public List getTopProducts() {
String key = "leaderboard:weekly:" + currentWeek();
Set> top = redisTemplate.opsForZSet()
.reverseRangeWithScores(key, 0, 9); // top 10 by score DESC
return top.stream()
.map(t -> new ProductScore(t.getValue(), t.getScore().longValue()))
.toList();
}
```

08-08 ElastiCache Redis Scaling and Failover

ElastiCache handles automatic failover: if a primary shard node fails, the replica is promoted within 60 seconds with no data loss (Redis replication is synchronous within a shard). For scaling, you can add shards (more keyspace capacity) or resize nodes (more memory per shard). ShopFlow scales from 3 shards to 6 shards for Black Friday — doubling memory and throughput.

Scaling Type	How	When To Use	Downtime
Scale up (vertical)	Increase node type (cache.r6g.large → cache.r6g.xlarge)	Memory pressure: used memory > 75% of node capacity	Zero — ElastiCache does rolling upg
Scale out (horizontal)	Increase number of shards (3 → 6)	Throughput limit: CPU > 80% or network bandwidth reached	Zero — ElastiCache redistributes slot
Add replicas	Increase replicas per shard (1 → 2)	Read scaling: need more read throughput from replicas	Zero — replica syncs from prim

■ Scale ShopFlow Redis Before Black Friday

1. ElastiCache → Redis clusters → shopflow-redis-cluster → Actions → Modify
2. Shard count: 3 → 6 (doubles keyspace capacity and throughput)
3. Apply modifications: Apply immediately (not during maintenance window)
4. Click Modify — ElastiCache adds 3 new shards; redistributes keyspace slots
5. Monitor: cluster shows "Modifying" status for 5-10 minutes
6. After modification: "Available" status; all 6 shards visible
7. Application code: no changes needed; Lettuce cluster client auto-discovers new nodes
8. After Black Friday: scale back from 6 → 3 shards to save cost

08-09 Redis Pub/Sub — Real-Time Notifications

Redis Pub/Sub allows one publisher to broadcast messages to multiple subscribers instantly. ShopFlow uses Redis Pub/Sub for real-time order status updates: when order-svc changes an order to "shipped", it publishes to a Redis channel. All notification-svc instances (ECS tasks) subscribed to that channel receive the message and push a WebSocket notification to the customer's browser.

```
// Publisher: order-svc publishes order status change
@EventListener(OrderStatusChangedEvent.class)
public void onStatusChanged(OrderStatusChangedEvent event) {
String channel = "order-updates:" + event.getUserId();
String message = objectMapper.writeValueAsString(Map.of(
"orderId", event.getOrderId(),
```

```
"newStatus", event.getNewStatus(),
"timestamp", Instant.now().toString()
));
redisTemplate.convertAndSend(channel, message);
// All subscribers to "order-updates:user-123" receive this message
}
// Subscriber: notification-svc listens for order updates
@Component
public class OrderUpdateListener {
@Autowired WebSocketHandler webSocketHandler;
@RedisSubscribe(pattern = "order-updates:*")
public void handleOrderUpdate(String message, String channel) {
String userId = channel.substring("order-updates:".length());
webSocketHandler.sendToUser(userId, message);
// Pushes to customer browser via WebSocket
}
}
```

■ Redis Pub/Sub does not persist messages — if notification-svc is not subscribed when the message is published (e.g., during a restart), the message is lost. For reliable messaging, use SQS or SNS instead. Use Redis Pub/Sub only for ephemeral real-time notifications where it is acceptable to miss occasional messages.

08-10 Caching Strategy for ShopFlow Services

Each ShopFlow microservice has a different caching strategy based on how frequently data changes and how critical freshness is. This table is the definitive reference for what gets cached, where, and for how long.

Service	Data Cached	Cache Key	TTL	Cache Pattern
product-svc	Product detail (name, price, images, category)	product:{productId}	300s (5 min)	Cache-Aside: @Cacheable; evict c
product-svc	Category product list (top 20 by rating)	category:{categoryId}:top	60s (1 min)	Cache-Aside: TTL-based invalidati
product-svc	Homepage featured products (curated list)	homepage:featured	3600s (1 hr)	Background refresh: job updates e
user-svc	User profile (name, email, address)	user:{userId}:profile	1800s (30 min)	Cache-Aside: evict on profile upda
order-svc	Order count per user (for "you have X orders")	user:{userId}:order-count	3600s (1 hr)	Write-through: increment on every
search-svc	Popular search queries and their results	search:{query}:results	120s (2 min)	Cache-Aside: short TTL (products
inventory-svc	Stock count per product (for "X left in stock")	inventory:{productId}:stock	30s	Write-through: decrement on purch

08-11 Redis Persistence — RDB Snapshots and AOF

By default, ElastiCache Redis does NOT persist data to disk — if all nodes restart, the cache is empty. For ShopFlow session tokens, an empty cache means all users are logged out simultaneously. ElastiCache provides two persistence options: RDB (point-in-time snapshots) and AOF (append-only file — logs every write command for replay on restart).

Persistence Type	How It Works	Recovery Guarantee	Performance Impact	ShopFlow Use
None (default)	Data only in memory; lost on restart	Zero — all data lost on restart	Best performance	Not suitable for sessions (wou
RDB (snapshots)	Periodic snapshot saved to S3 (every 1hr, 6hr, or 12hr)	Data loss = time since last snapshot (up to 1 hour)	Minimal — snapshot taken in background fork	Acceptable for product cache
AOF (append-only file)	Every write command appended to a log file, replayed on restart	Near-zero data loss (last second of writes may be lost)	10–30% throughput reduction	Required for session tokens: u

■ Enable AOF Persistence for Sessions
1. ElastiCache → Redis clusters → shopflow-redis-cluster → Modify
2. Enable AOF: Yes
3. AOF frequency: every-1-second (balance of durability and performance)
4. Backup snapshot: Enable → 1 day retention
5. Apply during: Apply immediately
6. Click Modify — cluster applies settings rolling (no downtime)

08-12 Redis Memory Management and Eviction Policies

When Redis memory reaches the configured maxmemory limit, it must evict (delete) keys to make room for new ones. The eviction policy determines which keys are evicted. Choosing the wrong policy causes silent data loss (important session tokens deleted) or write failures. ShopFlow sets maxmemory-policy carefully per use case.

Eviction Policy	Which Keys Are Evicted	Risk	ShopFlow Config
noeviction	No eviction — Redis returns error on new writes when full	High: application gets OOM errors, writes fail	NOT ShopFlow's choice — we never allow write failures on product
allkeys-lru	LRU (least recently used) across ALL keys	Low: discards old, unused data first	Used for general product cache clusters; old product data
volatile-lru	LRU only among keys WITH TTL set	Medium: keys without TTL never evicted; may fill memory	Safe if all keys have TTL; this becomes unsafe. Not ideal if some
volatile-ttl	Keys with shortest remaining TTL are evicted first	Low: keys closest to expiry evicted, least useful	Good for session cluster; evicts old sessions (about to expire) a
allkeys-lfu	LFU (least frequently used) across all keys	Low: evicts least popular data first	Better than LRU for product cache: evicts obscure produ

08-13 Monitoring ElastiCache — Key Metrics

ElastiCache publishes CloudWatch metrics for each shard. The most critical: CurrConnections, CacheHits/CacheMisses (compute cache hit ratio), Evictions (signals memory pressure), and ReplicationLag (replica is behind primary). ShopFlow monitors these and alerts when cache hit rate drops below 90%.

■ aws › CloudWatch › ElastiCache › shopflow-redis-cluster

ElastiCache Redis Key Metrics — Last 1 Hour

■ Performance metrics

■ CacheHits: 2,847,320 | CacheMisses: 85,419 | Hit Rate: 97.1% ■

■ GetTypeCmds latency (P99): 0.8ms | SetTypeCmds latency (P99): 1.1ms ■

■ Memory metrics

■ BytesUsedForCache: 9.8 GB / 13.07 GB per node = 75% ■■ (approaching alert threshold)

■ Evictions: 12,450 in last hour ■■ Some keys being evicted; consider scaling up node type

■ Connection metrics

■ CurrConnections: 142 across all nodes ■ (limit: 65,000 per node)

■ ReplicationLag: 0 ms ■ (replicas in sync with primaries)

■ Recommended actions

■ Memory at 75% with evictions: scale up to cache.r6g.xlarge (26 GB RAM) before Black Friday

---

08-14 Redis for Distributed Locks — Preventing Duplicate Orders

When a customer clicks "Place Order" twice quickly (double-click, network retry), two order creation requests can arrive simultaneously. Without a distributed lock, two identical orders could be created. Redis SETNX (Set if Not Exists) creates an atomic distributed lock: only one request gets the lock; the duplicate request sees the lock and returns the already-created order.

```
// Distributed lock using Redis (Redisson library)
@Service
public class OrderService {
@Autowired RedissonClient redisson;
public Order placeOrder(String userId, OrderRequest request) {
// Lock key = user + cart hash (prevents same user placing same cart twice)
String lockKey = "order-lock:" + userId + ":" + request.getCartHash();
RLock lock = redisson.getLock(lockKey);
try {
// Try to acquire lock; wait max 100ms; lock held for max 10s
boolean acquired = lock.tryLock(100, 10000, TimeUnit.MILLISECONDS);
if (!acquired) {
// Another request is placing the same order — return existing order
return orderRepository.findByUserIdAndCartHash(userId, request.getCartHash());
}
return createOrderInternal(userId, request);
} finally {
if (lock.isHeldByCurrentThread()) lock.unlock();
}
}
}
```

08-15 Hands-On Lab — Implement Product Cache with Redis

This lab adds Redis caching to product-svc, reducing Aurora query load by 95%. You will create the ElastiCache cluster, add Spring Cache annotations, test cache behavior, and verify cache hit metrics. Estimated time: 25 minutes.

Step	Action	Verification
1	Create ElastiCache Redis cluster (dev: cache.t3.micro, 1 shard, no replicas, no TLS)	ElastiCache console → shopflow-redis-dev → Status: Available
2	Add dependencies to pom.xml: spring-boot-starter-data-redis and spring-boot-starter-cache	mvn compile succeeds
3	Add @EnableCaching to the Spring Boot main class	Application starts with "Caching enabled" in logs
4	Configure application-dev.yml with Redis endpoint (from ElastiCache console)	spring.data.redis.host points to ElastiCache endpoint
5	Add @Cacheable("products") to ProductService.getProduct()	Code added; compile succeeds
6	Start application; call GET /api/products/prod-001 twice	First call: "Cache miss" in Aurora slow query log. Second call: < 1ms; no Aurora query
7	Update product in Aurora: PUT /api/products/prod-001 with @CacheEvict	Update succeeds; subsequent GET fetches fresh data from Aurora
8	Check CloudWatch: ElastiCache → CacheHits + CacheMisses	After 10 test calls: 8 hits (80%); real production would be 95%+ after warm-up
9	Check Redis key in console: DynamoDB → No, use redis-cli or ElastiCache console	Key "products::prod-001" visible with 5-minute TTL

08-16 Redis Cluster vs Single-Node vs Sentinel

ElastiCache Redis offers three deployment topologies. Choosing the right one for each environment balances cost, availability, and complexity.

Topology	Nodes	HA	Max Memory	Cost/month	ShopFlow Use
Single node (dev/test)	1 primary; no replica	None (data lost if node fails)	Up to 300 GB (single node)	$50 (cache.r6g.large)	Dev and test environments only
Cluster Disabled + 1 Replica	1 primary + 1-5 replicas	Yes: replica promoted on primary failure (~30s)	Up to 300 GB (single shard)	$100 (2× cache.r6g.large)	Small production apps; simple c
Cluster Mode Enabled	3-500 shards; 0-5 replicas per shard	Yes: per-shard failover; surviving shards unaffected	Scales to petabytes (add shards)	$300+ (6× cache.r6g.large = 3 shards × 2 nodes)	ShopFlow production: 3 shards

■ Cluster Mode Requirement: All keys in a hash tag must map to the same slot

Redis Cluster assigns 16,384 hash slots across shards. MGET and transactions (MULTI/EXEC) only work if all keys are on the same shard. Use hash tags to force related keys to the same shard: {user-123}:profile and {user-123}:orders both go to the same shard (hash computed on "user-123" only, not the full key). Without hash tags, MGET across different shards returns an error.

08-17 Redis Pipeline and MGET — Batch Operations

Every Redis command is a network round trip (~0.5ms). Fetching 10 product details separately = 10 × 0.5ms = 5ms of network overhead. Redis pipelines send multiple commands in one network round trip. MGET fetches multiple keys in one command. ShopFlow uses both to minimize Redis latency when rendering product lists.

```
// MGET: fetch multiple products in one Redis round trip
@Service
public class ProductCacheService {
@Autowired RedisTemplate redisTemplate;
public List getProducts(List productIds) {
// Build keys: ["product:prod-001", "product:prod-002", ...]
List keys = productIds.stream()
.map(id -> "product:" + id)
.toList();
// MGET fetches all keys in ONE Redis round trip (not N round trips)
List cached = redisTemplate.opsForValue().multiGet(keys);
// Find cache misses; fetch from Aurora in one batch query
List misses = new ArrayList<>();
for (int i = 0; i < productIds.size(); i++) {
if (cached.get(i) == null) misses.add(productIds.get(i));
}
if (!misses.isEmpty()) {
List fromDb = productRepository.findAllById(misses);
// Cache the misses for future requests
```

```
fromDb.forEach(p -> redisTemplate.opsForValue()
.set("product:" + p.getProductId(), p, Duration.ofMinutes(5)));
}
return mergeCachedAndFetched(cached, fromDb, productIds);
}
}
```

08-18 Redis for Feature Flags

Feature flags let ShopFlow enable or disable features for specific users or percentages of traffic without deploying new code. Redis is a fast, centrally-accessible store for feature flag state. All ECS tasks check Redis for flag status — flag changes propagate to all instances in < 1 second, without redeploying any service.

Feature Flag	Redis Key	Value	Use Case
New checkout flow	flag:new-checkout	enabled|disabled|25% (percentage rollout)	Gradually roll out redesigned checkout to 1% → 10%
Black Friday mode	flag:black-friday	enabled|disabled	Enable extra caching, disable non-essential services,
Maintenance mode	flag:maintenance-mode	enabled|disabled	Redirect all traffic to maintenance page without CDK d
A/B test: price display	flag:show-original-price	test-a|test-b	50% of users see original price crossed out; 50% do n

```
// Check feature flag in Spring Boot
@Service
public class FeatureFlagService {
@Autowired RedisTemplate redisTemplate;
public boolean isEnabled(String flagName, String userId) {
String value = redisTemplate.opsForValue().get("flag:" + flagName);
if ("enabled".equals(value)) return true;
if ("disabled".equals(value) || value == null) return false;
// Percentage rollout: "25%" means enabled for 25% of users
if (value.endsWith("%")) {
int pct = Integer.parseInt(value.replace("%", ""));
return Math.abs(userId.hashCode() % 100) < pct;
}
return false;
}
}
```

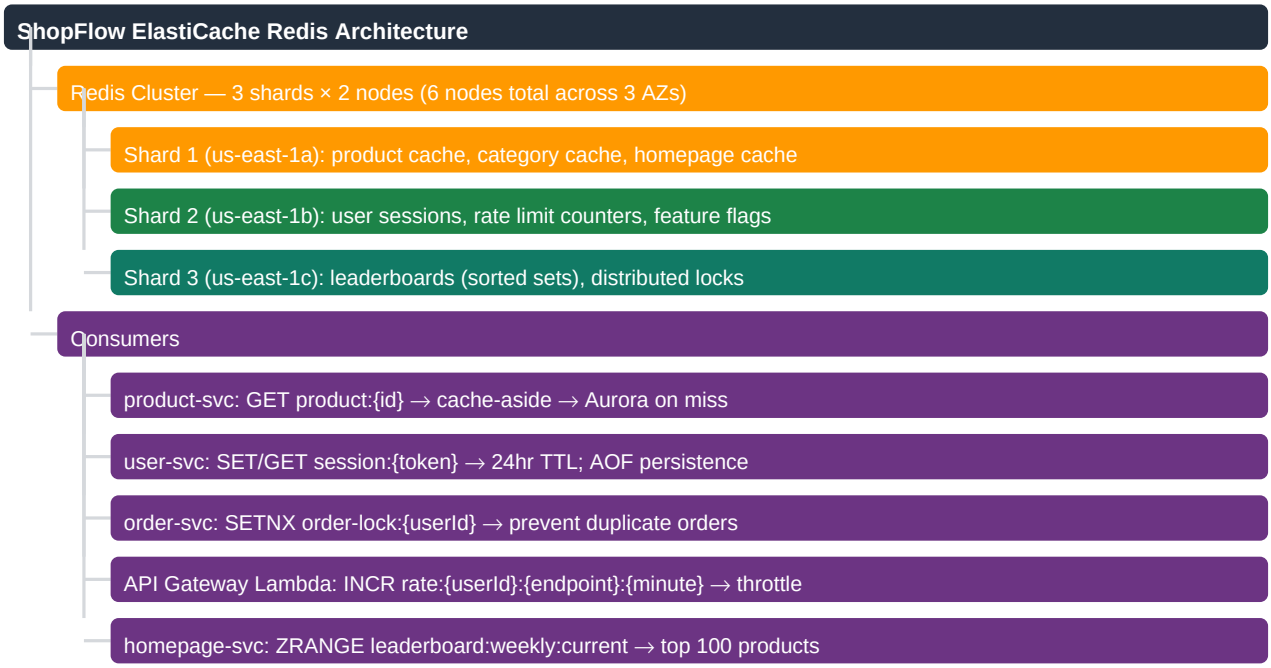
08-19 ElastiCache Cost Analysis

ElastiCache Redis is a significant ShopFlow cost center. Understanding the cost breakdown helps you right-size and justify the investment with concrete savings numbers.

Cost Item	Configuration	Monthly Cost	Value Delivered
Redis cluster nodes (3 shards × 2 nodes)	6 × cache.r6g.large × $0.156/hr × 730hr	$683/month	Reduces Aurora to 5% load; enables Aurora to be db.r6
ElastiCache backup storage	3 days × 78 GB	~$5/month	PITR for session/cache data
Data transfer (VPC internal)	Same AZ: free; cross-AZ: $0.01/GB × 50 GB/month	$0.50/month	Minimal; ECS tasks configured to prefer same-AZ Redi
Total ElastiCache cost		~$689/month	Saves $200/month in Aurora costs + enables 97% cach
Without Redis (Aurora only)	Need db.r6g.2xlarge writer + 2 readers to handle 1,000 req/s	$880/month	Much higher Aurora cost; 50ms latency vs < 1ms; wors

08-20 Redis Architecture Summary for ShopFlow

Here is the complete Redis architecture: which services use Redis, what they cache, and how the data flows.



Metric	Current	Target	Monitoring
Cache hit rate	97.1% (product cache)	> 95%	CloudWatch: CacheHits/(CacheHits+CacheMisses); aler
P99 latency	0.8ms GET, 1.1ms SET	< 2ms	CloudWatch: GetTypeCmds/SetTypeCmds latency P99;
Memory used	75% (9.8/13.07 GB per node)	< 75%	CloudWatch: DatabaseMemoryUsagePercentage; alert >
Evictions	12,450/hr (due to 75% memory)	0	CloudWatch: Evictions > 0 alarm; indicates memory pres

08-21 Redis Keyspace Notifications — React to Expiry

Redis keyspace notifications let your application subscribe to events: when a key is set, deleted, or expires. ShopFlow uses expiry notifications on session tokens — when a session expires (TTL reaches zero), notification-svc receives an event and logs the session termination for the security audit trail. This is different from Pub/Sub (Pub/Sub requires active publisher; keyspace notifications are triggered by Redis itself).

```
# Enable keyspace notifications in Redis (ElastiCache parameter group)
# notify-keyspace-events = "Ex" (E = keyevent events; x = expired events)
# This broadcasts when any key expires
// Java: subscribe to session expiry events
@Component
public class SessionExpiryListener implements MessageListener {
@Override
public void onMessage(Message message, byte[] pattern) {
String expiredKey = message.toString();
// expiredKey = "session:abc-123-def-456"
if (expiredKey.startsWith("session:")) {
String sessionId = expiredKey.substring("session:".length());
// Log session expiry for security audit trail
auditLogger.logSessionExpired(sessionId, Instant.now());
}
}
}
// Register: subscribe to __keyevent@0__:expired channel
```

■ Keyspace notifications add CPU overhead (Redis must track all key events). Only enable the specific event types you need (Kx for expired events only) rather than enabling all events (KEA). Check CPU utilization after enabling — if CPU increases significantly, the overhead may not be worth the benefit for your use case.

08-22 Redis Georadius — Location-Based Features

Redis GEO commands store latitude/longitude coordinates and support radius queries. ShopFlow uses Redis GEO for "show products available for same-day delivery in my area" — sellers register their warehouse locations in Redis, and the feature filters products by proximity to the customer's address.

```
// Store warehouse locations in Redis GEO
// GEOADD warehouses longitude latitude member
redisTemplate.opsForGeo().add("warehouses",
new RedisGeoCommands.GeoLocation<>(
"warehouse-nyc-001",
new Point(-74.006, 40.7128) // NYC longitude, latitude
)
);
redisTemplate.opsForGeo().add("warehouses",
new RedisGeoCommands.GeoLocation<>(
"warehouse-la-001",
new Point(-118.2437, 34.0522) // LA
)
);
// Find warehouses within 50km of customer location
GeoResults> nearby = redisTemplate.opsForGeo()
.search("warehouses",
new Circle(new Point(-73.95, 40.65), new Distance(50, Metrics.KILOMETERS)),
RedisGeoCommands.GeoSearchCommandArgs.newGeoSearchArgs().includeDistance()
);
// Returns: warehouse-nyc-001 (12.5 km away) — products from this warehouse available for
same-day
```

08-23 Redis Security Hardening Checklist

Redis requires careful security configuration. By default, Redis has no authentication, no encryption, and is accessible to anyone who can reach the network. ElastiCache enforces security via VPC, but additional hardening is required.

Security Control	Console Location	ShopFlow Setting
VPC isolation (no internet access)	ElastiCache cluster → VPC → Security groups	shopflow-redis-sg: inbound 6379 from shopflow-ecs-sg
Encryption in transit (TLS)	ElastiCache → Modify → Encryption in transit	Enabled; HTTPS-equivalent for Redis; negligible latency
Encryption at rest	ElastiCache → Modify → Encryption at rest	Enabled; AWS KMS key; protects snapshot data
Authentication token	ElastiCache → Modify → Auth token	AUTH token required; 128-character random string store
Redis version up to date	ElastiCache → Engine version	Redis 7.x (latest); auto minor version upgrades enabled
Backup enabled	ElastiCache → Backup → Enable backups	1-day retention minimum; 3-day for session data (AOF +
CloudWatch alarms	CloudWatch → Alarms	Alarms on: Evictions > 0, CurrConnections > 500, Replic

08-24 Redis Full-Text Search — RedisSearch

Redis Stack (available in ElastiCache 7.x) includes RedisSearch — a built-in full-text search engine. ShopFlow evaluates RedisSearch as a lighter-weight alternative to OpenSearch for autocomplete and simple product search. For simple use cases (autocomplete, prefix matching), RedisSearch can replace OpenSearch entirely at much lower cost.

Capability	RedisSearch	OpenSearch
Full-text search	Yes: tokenization, stemming, fuzzy matching, ranking by TF-IDF	Yes: sophisticated NLP, custom analyzers, relevance tuni
Autocomplete (prefix)	Yes: AUTOCOMPLETE command; extremely fast (< 1 ms)	Yes but requires prefix query + significant compute
Aggregations	Basic: GROUP BY, COUNT, SUM	Advanced: complex analytics, faceted search, histogram

Capability	RedisSearch	OpenSearch
Data storage	Redis keyspace: tight integration with existing Redis data	Separate service: own nodes, own storage, own management
Cost	No additional cost (part of ElastiCache Redis 7.x)	~$300-500/month for 3-node OpenSearch cluster
ShopFlow recommendation	Use RedisSearch for autocomplete only; keep OpenSearch for full product search (needs advanced relevance tu	

08-25 Redis Streams — Lightweight Message Queue

Redis Streams (XADD, XREAD, XGROUP) provide a persistent, append-only log with consumer groups — similar to Kafka but simpler, built into Redis. ShopFlow uses Redis Streams for lightweight event passing between ECS tasks within the same service cluster, where SQS would add unnecessary latency and cost for high-frequency, low-importance events.

Redis Streams Feature	Description	ShopFlow Use
XADD	Append message to stream; assigns auto-generated ID (timestamp-sequence)	order-service appends every order event: {orderId, userId, total, timest
XREAD / XREADGROUP	Read messages from stream; consumer groups allow multiple consumers to share load	notification-service consumer group: 3 ECS tasks share the stream; ea
Message acknowledgment	XACK marks message as processed; unacknowledged messages can be retried	If notification-service crashes mid-processing, message is reassigned
Stream retention	Limit stream length: MAXLEN 1000 (keep only last 1,000 messages)	Stream: MAXLEN ~ 10,000; streams auto-trim; no unbounded m

Comparison	Redis Streams	Amazon SQS
Latency	< 1ms	1-10ms
Throughput	Millions of messages/second (in-memory)	3,000 msg/sec standard; 300K msg/sec FIFO
Durability	AOF or RDB; data in memory — not ideal for financial events	Durably stored redundantly across 3 AZs; guaranteed delivery
Cost	Part of ElastiCache (already running)	$0.40/million requests
ShopFlow use	Analytics events, view counts, ephemeral real-time events	Order processing events, payment events — durability required

08-26 Performance Tuning — Reducing Redis Latency

ShopFlow targets P99 Redis latency < 2ms. Here are the tuning techniques that keep latency low under load.

Tuning Area	Default	ShopFlow Setting	Impact
Connection pool max-active	8	16 per service instance	Prevents connection wait under high concurrency; each of 8
TCP keepalive	0 (disabled)	60 seconds	Prevents stale connections that look open but are actually c
Read from replica for reads	No (read from primary)	Yes for product cache reads	Distributes read load across 2 nodes per shard; primary ha
Pipelining for bulk ops	No (one command per round trip)	Yes for MGET/MSET	Reduces 10 round trips to 1 for product list fetches
Avoid large keys	No enforcement	Alert if key > 1 MB	Large values increase serialization time; compress JSON a
CPU pinning on Redis nodes	None	cache.r6g: Graviton2 ARM	Graviton2 Redis nodes show 20-30% better throughput vs x

08-27 Black Friday Redis Preparation Checklist

Redis is the highest-risk single point of failure during Black Friday — if the cache layer goes down, Aurora receives 20x its normal traffic and likely cascades into a full outage. This checklist ensures Redis is ready.

Check	How to Verify	Pass Criteria	Risk if Fails
Memory < 60% (not 75%)	CloudWatch: DatabaseMemoryUsagePercentage	Below 60% with normal traffic; leaves 40% headroom for spikes	Evictions begin under load; cache hit rate dro
All 3 shards healthy	ElastiCache console → Cluster → all nodes green	6 nodes Available; 0 nodes unavailable	One shard down = 1/3 of cache unavail

Check	How to Verify	Pass Criteria	Risk if Fails
AOF + backups enabled	ElastiCache → Backup tab	AOF: enabled; automatic backup enabled (3-day retention)	Cache cold start = all sessions lost = all us
Connection pool sized for peak	Application config: lettuce.pool.max-active	max-active × services × tasks ≤ 65,000 (ElastiCache node limit)	Connection exhaustion under peak load; 5
Eviction policy set	Redis CONFIG GET maxmemory-policy	allkeys-lfu (evict least popular, not oldest)	Wrong policy evicts sessions during Bla
Runbook tested	Ops runbook: how to scale Redis in 10 minutes	Scale shard count 3 → 6 tested during staging; EFA	During Black Friday, scaling under pres

08-28 Redis Connection Best Practices

Connection management is the most common Redis performance issue. Too few connections cause queuing under load. Too many exhaust ElastiCache connection limits. Idle connections waste memory. These are the production-proven settings for ShopFlow.

Setting	Wrong Value	Correct Value	Why
max-active (pool size)	200 (over-provisioned)	16 per service instance	ElastiCache node limit: 65,000 connections. 10 service t
connection-timeout	30,000ms (30 seconds)	500ms	Fail fast: if Redis is unreachable, surface the error imme
command timeout	None (wait forever)	1,000ms	Single slow Redis command should not block a request
read-from-replica	false (always primary)	true for reads in product-svc	Distributes load; primary handles writes; replicas handle
Min-idle connections	0 (lazy — create on demand)	2 per pool	Keeps 2 warm connections; eliminates connection setup

■ Circuit Breaker for Redis

If Redis becomes unavailable, requests queue waiting for connections. Without a circuit breaker, this cascades into a thread pool exhaustion and complete API outage. ShopFlow implements a circuit breaker (Resilience4j): if Redis fails for 5+ consecutive requests in 10 seconds, open the circuit — serve responses without caching (fall through to Aurora) until Redis recovers. Better to be slow than completely down.

08-BP Best Practices and Anti-Patterns

✓ Always enable encryption in transit (TLS) and at rest for ElastiCache — Redis on plain TCP exposes credentials and cached data to network interception; TLS adds < 1ms overhead and is non-negotiable for production

✓ Set TTLs on all cached items — never cache without a TTL; stale data lives forever without it. For product details: 5-minute TTL. For session tokens: 24-hour TTL. For rate limit counters: 60-second TTL. TTLs prevent memory exhaustion and ensure eventual consistency with the source of truth.

✓ Use Redis Cluster mode for production — single-node Redis is a single point of failure and limits you to one node's memory. Cluster mode with 3 shards + 1 replica each = 6 nodes for HA; survives one full shard failure without data loss

✓ Monitor used memory vs max memory — alert when any shard exceeds 75% memory. Redis does not gracefully handle memory exhaustion; it starts evicting keys (or refusing writes) with no warning to the application. Stay below 75% to have headroom for traffic spikes.

✓ Use connection pooling in the Redis client — each application thread should not open its own Redis connection. Lettuce pool with max-active=16 handles 16 concurrent requests per pool. Without pooling, 100 concurrent requests open 100 Redis connections, exhausting the connection limit.

■ Do not store sensitive user PII in Redis unencrypted — session data should contain only user_id and roles, not full user profiles with addresses, payment info, or other PII. If Redis is compromised, minimizing cached data minimizes exposure.

■ Do not use Redis as the primary store for critical data — Redis is a cache; treat it as lossy. If ElastiCache restarts, data is lost (unless persistence is configured, which adds latency). Always write to Aurora or DynamoDB first; Redis is the fast cache layer, not the source of truth.

08-QR Quick Reference and Interview Q&A

Question	Answer
What is the difference between ElastiCache Redis and Memcached?	Redis: single-threaded, rich data structures (sorted sets, lists, hashes), persistence support, pub/sub,
How does ElastiCache Redis handle a primary node failure?	ElastiCache continuously monitors node health. When a primary node becomes unavailable: (1) Elast
What is cache stampede and how do you prevent it?	Cache stampede (thundering herd): a popular cache key expires; 1,000 concurrent requests see a cac